

Gossip-Scaled Semantic Forwarding in Named Data Networking

Sajjad Taghizadeh, Mohammad Mahdi Yari, Mohammad R. Shakournia, Pooya Jamshidi, Shahpour Rahmani

School of Electrical and Computer Engineering

University of Tehran, Tehran, Iran

{sajjad.taghizadeh, mohammadmahdiyari, shakournia}@ut.ac.ir

{pooya.jamshidi, shahpour.rahmani}@ut.ac.ir

Abstract—Named Data Networking drops a request whose name does not match a router’s forwarding table, so two hospitals naming the same sensor differently cannot reach each other’s data. Matching names by embedding similarity fixes this, but puts a sentence encoder in the forwarding path on every miss, and a per-router cache does not amortise it: split the same traffic across more routers and each cache sees less, so over a short run the network calls the encoder more often as it grows, not less. We share each resolved mapping over gossip instead, once, and only after a returned Data packet confirms it. Over a 60 s run at 16 edge routers this holds encoder-call growth to +9.9% against +59.6% for a per-router cache. The saving has limits: it falls to 6.5% over 600 s and reverses below about four edge routers. The same verification step yields two more results. The feedback a router learns from is biased, not just noisy: a producer’s refusal cannot tell “this name is not mine” from “this name is mine, but not in that wording”, a distinction NDN’s Network NACK was never built to carry. And a gossiped mapping can be checked, so a router can audit its peers from verdicts it already has, which attenuates a persistent poisoning attacker by 1.2× to 2.0×, more as the attack grows. That is an attenuation and not a hard bound: the rate-independence a bound predicts was tested, and failed.

Index Terms—Named Data Networking, semantic forwarding, anti-entropy gossip, conformal prediction, Byzantine resilience.

I. INTRODUCTION

Named Data Networking (NDN) [1], [2] forwards on names. A name that does not match the Forwarding Information Base (FIB) gets dropped. A client asking for /hospital/temp-sensor/room-101 gets nothing from a network that advertises /smart-hospital/floor-1/temperature/room-101, though the data is one hop away. Amadeo *et al.* [3] fix this by embedding names and matching them with cosine similarity, caching each resolution in an *Embedding Store* at the router that made it.

That design has a scaling problem their own conclusion leaves as future work. A per-router cache learns only from the traffic that router sees. Add edge routers without adding traffic and every cache sees a thinner slice, so the network calls the encoder more often, not less: +59.6% more inferences from 1 to 16 edge routers on one catalog and +67.4% on the other, for the same offered load.

Our premise is that the fix is a shared cache, not a bigger one, and that NDN already supplies what makes sharing safe. The synchronisation NDN has does not answer the question semantic forwarding raises: when is a learned name-to-route

mapping safe to share? Share a wrong resolution and a local recognition error becomes a network-wide routing error; wait for certainty and approximate matching has no point left. The question is therefore not how to disseminate state but how to admit it. A semantic match is a hypothesis and a returned Data packet is the producer confirming it, so gossiping only confirmed mappings keeps an unsupported guess out of shared state. That is what lets the similarity threshold stay permissive without the usual cost.

Contributions. We make three claims, each scoped to what we measured.

- *Sharing, not caching, is what keeps recognition cost from growing with the network (§IV).* We also measure what bounds this: the advantage shrinks with run length and reverses below about four edge routers.
- *The free labels a forwarding plane calibrates on are biased, not just noisy (§VI),* and the bias traces to a specific gap in the protocol rather than to noise. We found this defect in our own producer model, fixed it, and report where the fix helps and where it does not.
- *A verifying forwarding plane can audit its own peers (§VII).* Byzantine-robust gossip normally cannot catch a liar, only bound the damage. A gossiped mapping, unlike a gradient, can be checked against what its producer actually says.

What we do not claim. Edge tagging—marking an Interest with its resolved prefix so later hops skip the encoder—is Amadeo *et al.*’s, inherited here. Anti-entropy synchronisation is a decade old in NDN [4], [5]; our gossip layer carries a different payload over it. Learning per-item boundaries under an error budget is what vCache [6] does for prompt caching. We tried an error budget in place of the similarity threshold; it lost to a threshold tuned on one catalog and carried to another, and we report that as a withdrawn claim (§V).

II. BACKGROUND AND RELATED WORK

Semantic forwarding in ICN. Fuzzy Interest Forwarding [7] matches name components with Word2Vec and is the original statement of this problem. Amadeo *et al.* [3] use a sentence encoder, settle on a threshold of 0.7, and evaluate a single access router. INF-NDN [8] routes topic tags through central supernodes, concentrating semantic resolution instead of sharing it. Raza *et al.* [9] pair semantic name matching

with a learned network, but for FIB distribution and retrieval: their target is prefix compactness and lookup time, not the multi-router encoder cost measured here. Amadeo *et al.* [10] extend semantic ICN matching to service provisioning across cooperating digital twins, a different coordination problem from sharing one resolution once it is proven.

A note on “SAF”. In NDN, SAF already denotes Stochastic Adaptive Forwarding [11]. We write “semantic forwarding” or name the authors instead. Our result files still use the key `saf`, for continuity with earlier work.

Synchronisation NDN already has. Our anti-entropy and rumour-spreading pair is the classical epidemic construction of Demers *et al.* [12], whose $O(\log N)$ propagation under random peer choice [13] Section IV-D preserves by keeping one uniform pick. ChronoSync [4] and its successors reconcile datasets by exchanging condensed digests, and NLSR [5] already floods name-prefix reachability over such a layer; Moll *et al.* [14] survey how these designs differ. Our contribution is not the mechanism but the payload and the precondition: a wording-to-route mapping shared only after a Data packet confirms it. Section IV measures an unbounded version of this sync layer, because “why not just announce the prefix” is the first objection this design invites.

Learned boundaries under an error bound. vCache [6] learns per-item thresholds under a user-set error bound for prompt caching; Finamore *et al.* [15] do error-controlled similarity caching in a networking setting. The boundaries are conformal in the sense of Vovk *et al.* [16], pooling calibration across parties is federated conformal prediction [17], and the drifting variant we evaluate follows adaptive conformal inference [18]. That a threshold tuned on one catalog need not hold on another is the covariate-shift problem Tibshirani *et al.* [19] treat formally. What is specific to NDN is that the labels come free: an LLM cache needs a judge model to check whether a hit was correct, which is the cost one was avoiding, while a forwarding plane is told either way.

Biased feedback. Section VI is an instance of missing-not-at-random feedback, the pattern Saito *et al.* [20] study in implicit-feedback recommendation. A route’s label depends on a producer’s declared vocabulary, and that same vocabulary decides whether the request could have been served at all. Our contribution is finding this in a forwarding plane and naming the protocol gap that causes it.

Byzantine-robust gossip. Robust aggregation, clipping, trimming, a bounded per-peer contribution, bounds an adversary’s influence without ever identifying it. In gossip learning, a neighbour ships a gradient no honest node can check. Section VII exploits a difference: a gossiped mapping, unlike a gradient, can be checked against what its producer actually says.

III. SYSTEM MODEL AND METHOD

A. Simulator

We embed names with MiniLM [21], a distilled Sentence-BERT [22] model and the one the baseline uses. Each router has a single-server FIFO queue, so a strategy that adds work

to the forwarding path pays in queueing delay, not in a fixed constant. A strategy is consulted at one point, the FIB miss, and told the outcome afterwards; Content Store, PIT and exact FIB lookup are identical across strategies, so a difference in results is a difference in policy. The simulator is a discrete-event kernel in Python, with no external network simulator.

Similarity may choose a route, never decide that two names mean the same object. A semantic hit in the Content Store would hand a client data it did not ask for; one in the PIT would answer down the wrong chain.

B. Costs are measured, not assumed

Every latency constant is measured on the machine running the study and tagged as measured or assumed. One MiniLM-L6 inference costs 7.045 ms and a cosine search over the 50-entry FIB costs 0.0045 ms, a ratio of about 1566:1, so *the encoder is the only cost that matters*: any scheme that speeds up the similarity search optimises a rounding error. Our mechanism targets how often the encoder runs, not how fast.

C. Producers are modelled, not oracular

Letting a producer consult ground truth would be the weakest assumption available, so each holds a *declared schema* instead: the instance it serves and the terms it answers to, as a service registry or an OpenAPI description would provide. Declarations are deliberately incomplete. At coverage 0.7 a producer leaves 29% of the wordings it can be asked by undeclared and refuses requests genuinely meant for it. The feedback channel is informative without being correct, which is the situation any deployment is in.

D. Workload and metrics

Requests arrive as a Poisson process over a Zipf ($\alpha = 1.2$) popularity distribution, each consumer population drawing from an overlapping but distinct slice of the catalog; without that overlap sharing would be trivially redundant or trivially useless. Each catalog holds 50 services, 300 rewordings and 350 distractors that no service can satisfy, so precision stays measurable.

Scoring is correctness-aware. Every Interest ends in one of four states, correct, misdelivered, rejected or timeout, judged against ground truth the routers never see. Distractors are supposed to be rejected, so a single accuracy number would reward a strategy that answers nothing; we report precision, recall and false-positive rate separately. Interest Satisfaction Rate (ISR) is the share of resolvable Interests answered correctly, the same quantity as recall, reported because it is the baseline’s metric.

E. Comparison basis

We reproduce the baseline’s evaluation setting rather than invent our own: the access-edge topology, one-way latencies (10 ms client-to-router, 20 ms router-to-provider), the 50-entry FIB, the 30–300 Interests/s range, the Zipf skew of 1.2, a disabled Content Store, and an unbounded PIT all come from

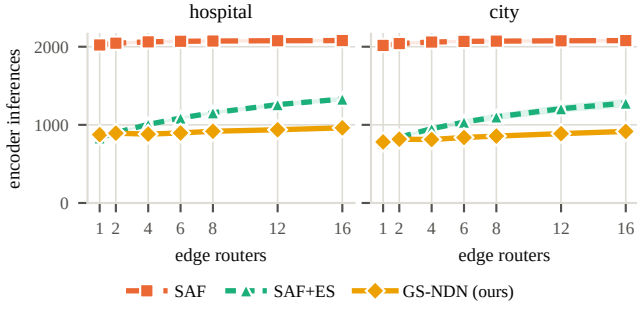


Fig. 1. Encoder invocations against edge routers. The per-router cache grows with the network; gossiped sharing stays nearly flat.

Amadeo *et al.* [3]. Our catalogs match their construction exactly: 50 services, one canonical name and six rewordings each, 350 satisfiable names, and an equal number of unrelated ones.

We report their metrics, mapped onto ours. Their *service discovery time* is our Interest-satisfaction time and decomposes the same way; their *processing time* is our encoder-invocation count times the measured inference cost, plus queueing. Their *share of Interests forwarded on the first attempt* is our satisfaction rate, with one deliberate difference: they count an Interest successful once it is forwarded and answered, while we also require the Data to come from the correct service. A confident match to the wrong room’s sensor is a success under their definition and a misdelivery under ours, and only ours can be tuned against that failure.

Absolute numbers are not comparable across the two papers, so we do not compare them. They report near-complete forwarding at $\text{Th} = 0.7$; ours yields recall of 0.866 and 0.804 there. The gap is in the rewordings, not the method: theirs are synonym substitutions, ours also drop hierarchy levels, reorder and abbreviate, and a seventh family draws on published IoT vocabularies. Every comparison here is between strategies inside one harness, never against a published number.

F. Cross-validation

A hand-written simulator can be internally consistent and still model NDN incorrectly, so we cross-check the transport layer, which is not our contribution, against ndnSIM on the same topology, delays and request pattern, with semantic resolution off. Median round-trip times agree to within 0.11 ms once the 1 ms producer service time our model charges is accounted for. All results are 20 seeded runs with 95% confidence intervals and reproduce across processes: three defects let interpreter-salted iteration order reach the event queue, and a regression test now compares two child processes run under differing PYTHONHASHSEED values.

IV. SCALING: SHARING VERSUS CACHING

Fig. 1 is the primary result. Semantic forwarding without a cache pays for every FIB miss, so it has nothing to erode

and grows +2.8% and +3.1% from 1 to 16 edge routers. The per-router cache thins as routers multiply and grows +59.6% on hospital and +67.4% on city. Sharing holds that to +9.9% and +17.3%, removing 27.7% and 28.3% of inferences at 16 edge routers. The claim is independent of any threshold or error budget: it is a property of how many times the encoder runs.

A. Two conditions that bound it

The horizon. A cold cache costs one inference per router per wording: N routers pay it N times, but they pay it once. Over 600 s instead of 60 s the per-router cache grows +14.4% rather than +55.2%, and gossip’s saving at 16 edge routers falls from 25.6% to 6.5% (city: 26.8% to 9.2%). The ordering never reverses, but the magnitude does: sharing pays a network that is large and still learning its catalog.

Network size. At a single edge router there is nobody to share with, and anti-entropy costs 4–5% more inferences than a plain per-router cache. The benefit appears from about four edge routers upward.

B. Is the decay an artifact of a closed catalog?

A natural hypothesis is that the decay is an artifact of a finite catalog: once every router has met all 300 rewordings there is nothing left to learn. We tested it by introducing wordings over the run instead of making them askable at $t = 0$. The hypothesis is false, and instructively so: a slower arrival rate lowers the saving at *every* horizon instead of sustaining it, from 25.6% (closed) to 3.0% (one new wording every 5 s) at 60 s. Gossip’s saving comes from the *backlog* of resolutions some routers hold and others do not, and a closed catalog at $t = 0$ is the largest backlog obtainable.

C. Would a more aggressive sync layer do better?

NDN already ships anti-entropy synchronisation, so the obvious objection is to turn it on and drop this layer. We bound that with an arm in which every router reconciles with every peer each round with no cap per exchange. It removes *fewer* inferences than the bounded protocol (21.0% against 25.6% at 16 edge routers) while sending 1.2–1.6 $\times$ the bytes. Anti-entropy at fanout 2 already reaches everyone in $O(\log N)$ rounds, and what limits the saving is how many distinct wordings the network has yet to see, which no amount of synchronisation changes. What does the work is *what* is shared and *when*, not how aggressively.

D. A Pareto frontier, not an operating point

The gossip period is a degree of freedom, and a single value would hide a trade worth seeing. Sweeping it from 0.25 to 5 s at 16 edge routers moves the saving from 24.4% to 18.0% while gossip traffic falls from 933 to 433 kB; at 4 edge routers the saving is flat across the same range (8.5–9.6%). The exchange rate between bytes and inferences therefore depends on network size, and a conclusion drawn at one operating point would have been wrong at the other. The chosen 0.5 s is Pareto-optimal among the points measured: 0.25 s is worse on *both* axes.

Delta-biased peer selection. Once converged, the modal exchange is two agreeing digests and no payload. Ranking partners by how far behind they are redirects those exchanges for free, keeping one uniform pick of `fanout` so the process stays epidemic. It is the only mechanism we tested that moves the frontier outward instead of along it, saving 11.1 ± 7.5 inferences at 0.5 s and 24.9 ± 7.3 at 1.0 s (20/20 seeds), though only 1–1.5% of invocations and only at mid-range sizes.

E. Latency and energy

Encoding on every FIB miss saturates the forwarding path: p95 round-trip time rises from 79 to 172 ms between 30 and 300 Interests/s, while cached and gossiped variants stay flat at 78–83 ms. The knee sits just past 200 Interests/s, reproducing the bottleneck the baseline reports beyond roughly 210 for the same encoder, which we did not tune for. On the radio model of Askar *et al.* [23], with encoder energy charged at measured CPU draw, GS-NDN uses 634 J against 735 J uncached and 654 J for a per-router cache, at higher satisfaction. A character n -gram control reaches 0.791 satisfaction where MiniLM reaches 0.969: it resolves abbreviations and fails on true synonymy, which is what the encoder pays for.

V. CALIBRATING FROM FREE LABELS

A threshold is a property of the catalog it was tuned on: 0.7 works on the baseline’s and yields recall of 0.866 and 0.804 on ours. One number must also serve every route. Some services sit alone in embedding space and a loose score is safe; others have near-identical siblings and a high score is a coin flip. Every semantic forwarding decision is an experiment the network answers, so (score, verdict) pairs accumulate for free, and an operator can set an error budget ϵ instead: for route r , the lowest score whose upper confidence bound on the error rate among observations at or above it stays within ϵ . We use a Wilson bound, since with a dozen observations the normal approximation gives a route with no errors a bound of exactly zero, and shrink each route toward a pooled boundary at a rate set by its own evidence.

Two results follow from this. Only one of them is good.

Risk control deadlocks without exploration. A boundary set too high blocks exactly the decisions that would produce the evidence to lower it. Left unexplored the controller settles at refuse-everything and stays there, reaching 0.732 satisfaction at $\epsilon = 0.02$; tightening ϵ changes nothing, since it is already refusing what it can. Spending 5% of refused decisions on evidence reaches 0.818. This is not an estimator bug but a property of learning from one’s own choices, and we report those exploratory forwards separately because they are decisions the budget did not cover.

The efficiency claim was tested and withdrawn. The budget holds out-of-sample in all fourteen tests: two catalogs crossed with seven error budgets from $\epsilon = 0.02$ to $\epsilon = 0.40$. But against a threshold tuned on one catalog and carried to the other, the controller Pareto-dominates in *none* of the fourteen while the transferred threshold dominates in seven. What survives is narrower: the controller needs no labelled

sample of the domain it runs on. An operator who has one should tune a threshold instead.

VI. THE FEEDBACK IS BIASED, NOT JUST NOISY

This is the finding the paper stands on, alongside the scaling result. It came from a defect we found in our own producer model.

A producer refusing a semantically-resolved Interest has two distinct reasons, and a plain refusal cannot tell them apart. “This name is not mine” is a routing fact: the mapping that sent the Interest here was wrong. “This name is mine, but not in that wording” is not: the route was correct, and only the phrasing fell outside what the producer declared. Collapsing the two makes every gap in a producer’s vocabulary look like evidence against a route that was right. HTTP separates 404 from 406 for this reason and DNS separates NXDOMAIN from NODATA. NDN’s Network Nack was never built to carry the distinction; its reason codes cover forwarding-plane conditions such as congestion or a missing route, not a producer’s semantic judgement. Semantic forwarding is what makes that judgement necessary.

The distinction has to be one a producer can make. Our first implementation tested whether the producer published *the prefix the router had chosen*. A misroute satisfies that test by construction, since that prefix is why the Interest was routed there. At full declaration coverage, where a genuine wording gap is nearly impossible, 132 of 157 refusals reported “unknown wording” about a route that was wrong, 2 were the case the label exists for, and the routing-error label never fired. The channel carried almost no information.

What a producer does know is which *instance* it is attached to: the room, the junction, the meter. A request that does not name that instance is not for this service, however it is worded. That test is exact in our runs: every instance mismatch was a real misroute or an unsatisfiable distractor, with no false positives. It turns a dead channel into an informative one. At coverage 0.5, 291 refusals are correctly labelled routing errors and 546 wording gaps, 71% of them genuine.

The residue inverts. A producer serving temperature readings in room 101, asked for humidity in that room, cannot tell a metric it does not serve from a synonym of its own it forgot to declare. The base rate of genuine gaps among such refusals moves from 0.021 at full coverage to 0.711 at half, and the right policy inverts with it. Ignoring ambiguous refusals gains 0.07 satisfaction at coverage 0.5 (0.844 against 0.774) and *costs* 0.014 at full coverage on city (0.963 against 0.978). Declaration coverage is a property of other organisations’ service registries, which an operator does not observe.

We tried to close this with inverse propensity weighting, estimating a propensity per route. It failed informatively: the estimated weight sat between 0.414 and 0.470 in *every* regime and on both catalogs, because a correct route at low coverage both serves often and is refused often, so a route-conditioned ratio barely moves. What does move is a single network-wide base rate. We report the regime inversion as an open problem

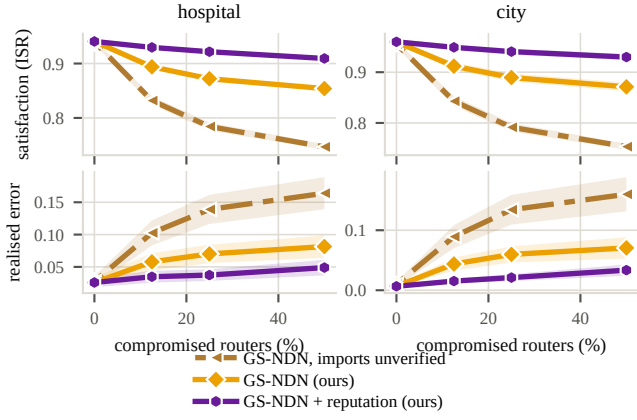


Fig. 2. Satisfaction and realised error under a persistent poisoning attacker. Reputation is free at zero compromise and recovers most of the attack’s cost thereafter.

whose magnitude we have measured, not as a mechanism that closes it.

A. Schema drift

Producer relocation hurts every strategy equally, since FIB withdrawal invalidates a stale mapping before any producer can refuse it. Schema drift is different: a producer quietly narrows what it answers to, with no routing event. That removes the confound, and there verification’s advantage appears: $+0.0042 \pm 0.0018$ satisfaction on hospital (16 of 20 seeds) and $+0.0124 \pm 0.0045$ on city (19 of 20), paired seed by seed. Nothing in the routing plane observes a schema drift; feedback is the only signal that can.

VII. THREAT MODEL AND PEER REPUTATION

A. Threat model

What the attacker controls. A fixed share of routers, chosen uniformly at random, each a full participant holding real keys and peer relationships, so nothing here is defeated by authenticating the channel. Each behaves correctly on the forwarding path and lies only into gossip. A router that drops traffic is a denial-of-service problem the ICN literature already treats [24] and would confound the measurement.

What it can do. It injects fabricated mappings and calibration evidence and re-injects both every round, so one retraction settles nothing. It knows the catalog well enough to target names clients request, which is the strong assumption here.

What it cannot do. Forge another router’s identity, partition the network, observe traffic it does not carry, or make a producer answer a name it does not publish. That boundary matters for §VI: the refusal is authored by the producer, so a compromised *router* cannot forge one. What a compromised *producer* could do is unmeasured.

What limits damage structurally. A router’s own confirmed mappings outrank anything it is told. That is not a defence added against this attack; it falls out of gossiping only what a returned Data packet confirmed.

TABLE I
SATISFACTION AND REALISED ERROR AGAINST COMPROMISED SHARE, 8 EDGE ROUTERS. “IMPORTS UNVERIFIED” IS THE BEHAVIOUR BEFORE WE EXTENDED VERIFICATION TO MAPPINGS LEARNED OVER GOSSIP.

compromised	0%	12.5%	25%	50%
<i>hospital, satisfaction</i>				
Imports unverified	0.940	0.832	0.784	0.747
GS-NDN	0.941	0.894	0.872	0.854
+ reputation	0.941	0.930	0.922	0.909
<i>hospital, realised error</i>				
Imports unverified	0.027	0.103	0.139	0.164
GS-NDN	0.026	0.058	0.070	0.081
+ reputation	0.026	0.035	0.037	0.049
<i>city, satisfaction</i>				
Imports unverified	0.956	0.843	0.791	0.753
GS-NDN	0.960	0.912	0.890	0.871
+ reputation	0.960	0.949	0.941	0.930
<i>city, realised error</i>				
Imports unverified	0.010	0.089	0.134	0.160
GS-NDN	0.007	0.044	0.060	0.071
+ reputation	0.007	0.015	0.021	0.033

B. Verification-grounded reputation

Byzantine-robust gossip normally defends without ground truth, bounding an adversary’s influence without ever identifying it. A gossiped mapping is different: it is falsifiable, and the producer answers it. We charge each refusal to the peer that supplied the mapping and never reinstall a pair the router disproved first-hand. Trust is a Wilson *lower* bound on a peer’s success rate, mirroring the upper bound on error in §V: both bound in the direction that costs us. A peer with no history is believed, deliberately, since distrusting unaudited peers would break the bootstrap gossip exists for.

Calibration evidence gets a different defence, since one observation about a route is not immediately falsifiable: no single peer may supply more than a quarter of a route’s calibration window, bounding its contribution by construction and not by detection.

Table I and Fig. 2 show the effect. At 50% compromise, reputation recovers 0.162 of the 0.194 satisfaction the attack costs on hospital (84%) and 0.177 of 0.207 on city (86%), and cuts realised error by 40% and 52%. Both are measured at the single injection rate used throughout this table and figure; §VII-C sweeps that rate and finds the recovered share is not constant, so these describe one operating point, not a guarantee. It costs nothing when nobody is attacking, 0.941 against 0.941, which is the number that decides whether it can be left switched on. It works for the stated reason: the count of distrusted peers rises linearly with the compromised share, reaching 29.3 at 50%.

C. An attenuation, not a breakdown bound

It is tempting to formalise the residual as a hard bound. Each pair of an honest router and a compromised neighbour admits at most k mappings before the liar is cut off, where k is the trust prior, giving admitted poison $\lesssim 2Ef(1-f)k$ for E adjacencies and compromised fraction f : quadratic in f , and independent of the injection rate.

We tested both halves. The mechanism half holds: sweeping k over $\{1, 2, 4, 8, 16\}$ moves surviving poison monotonically from 494 to 1067, so the admission fee is the knob. **Rate invariance fails.** Taking the attacker from 1 to 16 fabricated mappings per round raises the defended arm’s realised error by $1.43\times$ on hospital and $1.91\times$ on city. A bound counted in pairs should not move with rate; ours does, so we claim no breakdown point.

What survives is an attenuation whose value *grows* with attack intensity. The undefended arm rises $2.35\times$ and $3.25\times$ over the same range, so the ratio between defended and undefended widens from $1.24\times$ to $2.04\times$ on hospital and $1.72\times$ to $2.93\times$ on city. The defence is worth more the harder it is attacked.

The sweep also selects a better prior than intuition did. A fee of $k = 1$ is best under attack but on a clean network falsely distrusts 10.9 honest peers and degrades sharing by 3%; $k = 2$ distrusts 1.1 for 0.06%, while realising 0.037 error against $k = 4$ ’s 0.044. We use $k = 2$. The honest-network control is what makes that visible; the attack column alone would have argued for $k = 1$.

VIII. LIMITATIONS AND FUTURE WORK

Our synonyms are easier than standardised ones. Our catalogues use a lexicon we wrote. To test whether it flatters the encoder we added rewordings from Brick, SAREF, Project Haystack and W3C SSN/SOSA. They are markedly harder: on city, rank-1 recognition is 0.578 for ontology-derived wordings against 0.920 for ours. That confirms the concern instead of removing it, and every recognition number here should be read as an upper bound.

Provenance is not implemented. Our threat model names one other defence: signing an observation so a lie traces to its origin, not to whichever honest peer last relayed it. A router that penalises the peer a bad mapping arrived from cannot tell an originator from an honest relay, so §VII defends against a source, not the network that source hides behind. A signed per-hop provenance field on each gossiped mapping is the next step.

A compromised producer is unmeasured. Producers are trusted about their own services throughout; one that lies about its own schema sits outside §VII, which controls only routers.

The encoder guard is a trade, not a win. Refusing to pool calibration scores across routers running different encoders yields lower satisfaction *and* lower realised error than pooling naively (0.948/0.023 against 0.951/0.027 at a quarter heterogeneous). For an error-budget system that trade is defensible, but it is not a clean win.

IX. CONCLUSION

Sharing producer-confirmed resolutions through gossip, rather than caching them independently at each router, substantially limits recognition-cost growth as the network expands under the evaluated conditions. We identify the two factors that bound this benefit: network size and learning horizon. The same producer feedback signal that enables controlled sharing

also provides a source of calibration evidence and peer-audit information for the forwarding plane. However, this feedback is not simply noisy; it is biased because existing refusal signalling does not distinguish routing errors from unexpressed producer vocabulary. We show that correcting this bias can help only under regimes determined by declaration coverage, a quantity that may be unavailable to operators. Finally, we report two mechanisms that did not improve their intended objective under our evaluation, highlighting that understanding failure modes is as important as demonstrating successful designs.

REFERENCES

- [1] V. Jacobson *et al.*, “Networking named content,” in *Proc. ACM CoNEXT*, 2009.
- [2] L. Zhang *et al.*, “Named data networking,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, 2014.
- [3] M. Amadeo, S. Serrano, A. Molinaro, M. Nitti, and G. Ruggeri, “Enhancing IoT service discovery through semantic name-based forwarding,” *IEEE Internet Things Mag.*, vol. 9, no. 2, pp. 142–148, Mar. 2026.
- [4] Z. Zhu and A. Afanasyev, “Let’s ChronoSync: Decentralized dataset state synchronization in named data networking,” in *Proc. IEEE ICNP*, 2013.
- [5] A. K. M. M. Hoque *et al.*, “NLSR: Named-data link state routing protocol,” in *Proc. ACM SIGCOMM ICN Workshop*, 2013.
- [6] L. Schroeder *et al.*, “vCache: Verified semantic prompt caching,” *arXiv:2502.03771*, 2025.
- [7] K. Chan *et al.*, “Fuzzy interest forwarding,” in *Proc. AINTEC*, 2017.
- [8] G. M. Raza, I. Ullah, M. S. U. Din, M. A. U. Rehman, and B.-S. Kim, “INF-NDN IoT: An intelligent naming and forwarding in name data networking for Internet of Things,” *IEEE Access*, vol. 12, 2024.
- [9] G. M. Raza, T. T. Sari, G. Secinti, K. Aurangzeb, M. N. Ali, and B. S. Kim, “Semantic-driven intelligent name distribution & retrieval for named data networking,” in *Proc. IEEE SmartNets*, 2025.
- [10] M. Amadeo, G. Ruggeri, A. Molinaro, M. Nitti, and S. Serrano, “Service provisioning in digital twin networks with semantic-aware information centric networking,” in *Proc. IEEE CCNC*, 2026, work in progress.
- [11] D. Posch, B. Rainer, and H. Hellwagner, “SAF: Stochastic adaptive forwarding in named data networking,” *IEEE/ACM Trans. Netw.*, vol. 25, no. 2, 2017.
- [12] A. Demers *et al.*, “Epidemic algorithms for replicated database maintenance,” in *Proc. ACM PODC*, 1987.
- [13] R. Karp *et al.*, “Randomized rumor spreading,” in *Proc. IEEE FOCS*, 2000.
- [14] P. Moll *et al.*, “SoK: The evolution of distributed dataset synchronization solutions in NDN,” in *Proc. ACM ICN*, 2022.
- [15] A. Finamore *et al.*, “Error-controlled approximate-key caching,” in *Proc. IEEE INFOCOM*, 2022.
- [16] V. Vovk, A. Gammernan, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.
- [17] C. Lu *et al.*, “Federated conformal predictors for distributed uncertainty quantification,” in *Proc. ICML*, 2023.
- [18] I. Gibbs and E. Candès, “Adaptive conformal inference under distribution shift,” in *Proc. NeurIPS*, 2021.
- [19] R. J. Tibshirani *et al.*, “Conformal prediction under covariate shift,” in *Proc. NeurIPS*, 2019.
- [20] Y. Saito *et al.*, “Unbiased recommender learning from missing-not-at-random implicit feedback,” in *Proc. WSDM*, 2020.
- [21] W. Wang *et al.*, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Proc. NeurIPS*, 2020.
- [22] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP*, 2019.
- [23] N. A. Askar, A. Habbal, H. Hamouda, A. M. Alnajim, and S. Khan, “SEF: A smart and energy-aware forwarding strategy for NDN-based Internet of healthcare,” *Comput. Mater. Continua*, 2024.
- [24] C. Ghali, G. Tsudik, and E. Uzun, “Needle in a haystack: Mitigating content poisoning in named-data networking,” in *Proc. NDSS SENT*, 2014.